

CURSO 2: ALGORITMOS AVANZADOS Y ESTRUCTURAS DE DATOS

CLASE 08 • NIVEL 2 - INTERMEDIO

Clase 08: Recursividad y Programación Dinámica con Memoización

«Programación Dinámica como Recordar el Pasado para no Resolverlo Dos Veces»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 08** del **Curso 2: Algoritmos Avanzados y Estructuras de Datos**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

Objetivos de la Sesión

Competencia Conceptual: Comprender subproblemas superpuestos, subestructura óptima y memoización con `functools.lru_cache`.

Competencia Práctica: Optimizar algoritmos exponenciales $O(2^n)$ transformándolos en lineales $O(n)$.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 08: Recursividad y Programación Dinámica con Memoización

La programación dinámica es una técnica de optimización que almacena los resultados de subproblemas ya resueltos.

☀ Metáfora Central: Programación Dinámica como Recordar el Pasado para no Resolverlo Dos Veces

Si te pregunto cuánto es $1+1+1+1$, cuentas 4. Si añado otro $+1$, sabes que es 5 porque recordaste el 4 anterior.

Principios Teóricos y Modelo Mental

Recursión simple (Naive): Resuelve los mismos subproblemas miles de veces (explosión exponencial $O(2^n)$).

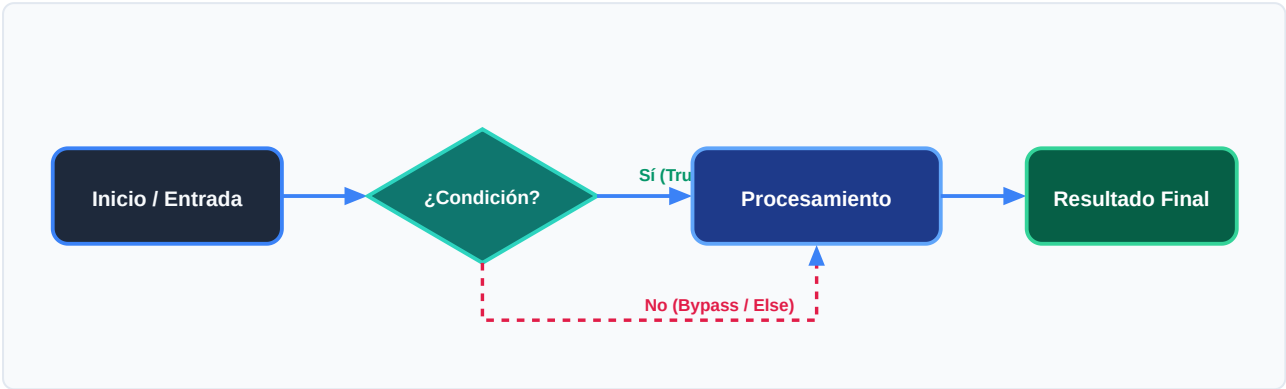
Memoización (Top-Down): Guarda en una tabla hash el resultado de cada llamada de función.

⚡ Regla de Oro en Python

En Python, decora tus funciones recursivas puras con `@functools.lru_cache(maxsize=None)`.

Arquitectura y Movimiento de Datos en Memoria

Poda del árbol de recursión mediante caché de resultados.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Llamada con argumentos n.	Búsqueda en el diccionario de caché.
2. Evaluación	Cache Hit: Retorno instantáneo $O(1)$.	Subárbol de cálculo omitido.
3. Transformación	Cache Miss: Ejecución y cálculo del valor.	Resultado computado.
4. Retorno / Salida	Almacenamiento en caché y retorno.	Caché actualizada.



Visualización Mental

La memoización convierte un árbol exponencial en una línea recta.

Código de Demostración en Python 3.10+

Optimización extrema de Fibonacci con @lru_cache:

main.py (Python 3.10+)

PEP 8 Compliant

```
from functools import lru_cache
import time

@lru_cache(maxsize=None)
def fibonacci(n: int) -> int:
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

t0 = time.perf_counter()
res = fibonacci(50)
t1 = time.perf_counter()

print(f"Fibonacci(50) = {res}")
print(f"Calculado en: {(t1 - t0)*1000:.4f} ms (Tiempo O(n))")
```

Análisis de Ingeniería del Código

Uso del decorador estándar lru_cache que almacena resultados en memoria C interna.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Límite de recursión en Python (RecursionError).

⚠ Gotcha Frecuente (Trampa de Principiante)

Recursiones muy profundas superan el límite de la pila de llamadas (por defecto 1000).

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
def contar(n):  
    if n == 0: return 0  
    return contar(n - 1) # ❌ Falla con n > 1000  
por RecursionError
```

✅ Patrón Correcto

```
# Enfoque iterativo (Bottom-Up) o  
sys.setrecursionlimit  
def contar_iterativo(n):  
    return sum(range(n)) # ✅ O(1) memoria
```

🛡 Consejo de Resiliencia en Producción

Siempre que puedas, traduce algoritmos recursivos a tablas iterativas (Tabulación).

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 08: Recursividad y Programación Dinámica con Memoización**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Programación Dinámica como Recordar el Pasado para no Resolverlo Dos Veces
2. Regla de Oro	En Python, decora tus funciones recursivas puras con <code>@functools.lru_cache(maxsize=None)</code> .
3. Gotcha Crítico	Recursiones muy profundas superan el límite de la pila de llamadas (por defecto 1000).
4. Buenas Prácticas	Siempre que puedas, traduce algoritmos recursivos a tablas iterativas (Tabulación).

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Resuelve el problema del cambio de monedas (mínimo número de monedas para formar un monto) usando DP.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.